Persisting GraphQL queries

- Tyk allows exposing a GraphQL query as a REST endpoint.
- Achieved using persist_graphql inside extended_paths.
- Configured on an HTTP-type API definition.
- Useful for:
    - Creating simple, cacheable REST endpoints
    - Predefined access to known GraphQL operations

Requirements:

- The `target_url` must point to a GraphQL upstream.
- `enable_context_vars` must be set to `true`.

Persisting GraphQL queries

API Definition & Versioning Setup

- Basic API definition:

```
"proxy": {
  "listen_path": "/trevorblades/",
  "target_url": "https://countries.trevorblades.com",
  "strip_listen_path": true
}
```

- Enable versioning with:
  - `use_extended_paths: true`
  - `not_versioned: true`
- GraphQL to REST middleware lives inside `extended_paths.persist_graphql`.

Tyk

Persisting GraphQL queries

The vital part of this is the `extended_paths.persist_graphql` **field**

Sample persist_graphql configuration:

```
"persist_graphql": [
  {
    "method": "GET",
    "path": "/getContinentByCode",
    "operation": "query ($continentCode: ID!) { continent(code: $continentCode)
{ code name countries { name } } }",
    "variables": {
      "continentCode": "EU"
    }
  }
]
```

Tyk

Persisting GraphQL queries

Each entry includes:

- method – e.g., `GET`
- path – e.g., `/getContinentByCode`
- operation – the GraphQL query
- variables – parameters passed to the query

```
{
    "data": {
        "continent": {
            "code": "EU",
            "name": "Europe",
            "countries": [
                {
                    "name": "Andorra"
                },
                ...
            ]
        }
    }
}
```

Tyk